Continuous Sign Language Recognition Model on Facially Occluded Signers

Ajmain Alam, Juan Jeong, Ky Dang, and Matthew Lee

Abstract Sign Language incorporates manual and nonmanual components, associating facial movements with hand signs in order to communicate words and phrases. Continuous sign language recognition (CSLR) models consider both factors when generating exact, glossed translations of sign languages. However, because of this physical requirement, many contemporary models struggle to effectively recognize signs from facially occluded signers. In order to address this, we present the FaceBranch, an augmentation of an existing CSLR model, CorrNet. The FaceBranch employs localized targeting of the eyes to enhance the model's recognition of occluded signers, before merging with a modified CorrNet model through late-fusion to translate signs. The FaceBranch improved Word Error Rate (WER) in signer-independent (SI) testing by roughly 3 percentage points for both nonoccluded and occluded signers, but showed no improvement in unseen-sentence (US) testing. Both improvements were determined in comparison to RGB CSLR models, including VAC, SMKD, TLP, SEN, CorrNet, Swin-MSTP, and SlowFastSign.

1 Introduction

Hearing loss affects hundreds of millions worldwide, with more than 5% of the global population requiring some form of rehabilitation [1]. Hearing impairments

Ajmain Alam
Princeton University, Princeton, NJ, e-mail: ajmain.alam@princeton.edu

Juan Jeong
Princeton University, Princeton, NJ, e-mail: juan.jeong@princeton.edu

Ky Dang
Princeton University, Princeton, NJ, e-mail: ky.dang@princeton.edu

Matthew Lee
Princeton University, Princeton, NJ, e-mail: ml1271@princeton.edu

can create complex communication barriers between hearing individuals and deaf or hard-of-hearing populations, causing difficulties in daily life for these communities [2].

There are hundreds of distinct, stand-alone regional sign language variants. These languages are primarily composed of combinations of manual signs, such as hand gestures, and nonmanual elements, such as facial expressions [3]. In order to dismantle these communication barriers, many researchers are actively working on artificial intelligence models that can recognize and translate various sign languages into spoken or written words, called sign language recognition (SLR). SLR models use computer vision and machine learning, analyzing sign language videos to return interpreted gloss translations of them [4]. Gloss translation is a linguistic tool that allows for the chronological ordering of sign language words. This provides support to the model in terms of semantic information associations and nuance through word clustering and relative meaning [5].

Two primary SLR models dominate the landscape: continuous sign language recognition (CSLR) and isolated sign language recognition (ISLR). These techniques differ primarily in their processing capabilities, with ISLR being only capable of translating one gloss per clip (one word or phrase), versus CSLR emphasizing translations of continuous signs into entire sentences [4, 6, 7].

Saudi Sign Language (SSL) is the primary sign language used within the Kingdom of Saudi Arabia, as well as several other parts of the Arabic-speaking world. A 2022 population and housing census estimated over 84,000 people living with some form of hearing impairment, representing 6.2% of the disabled population in Saudi Arabia [8]. Despite this, there remains a disparity in the funding and progression of AI-powered translation technologies geared towards SSL. Among the most prominent efforts is the Isharah dataset, an open-source resource used to train AI models for the recognition and translation of SSL in written language [9]. Importantly, the prevalence of facially occluding headwear, such as niqabs or face masks, within Arabic-speaking regions requires video datasets which include specific camera angles and lighting in order to successfully capture nonmanual elements that are crucial to effective sign-language recognition, a quality that the Isharah dataset addresses [10].

This research aims to build on the CorrNet AI model, using the Isharah-1000 dataset to improve sign language recognition for facially occluded individuals [9, 11]. This was achieved by modifying CorrNet and adding a separate facial recognition branch which specifically targets non-occluded areas.

2 Background and Related Work

2.1 CorrNet

A primary drawback of CSLR lies in its struggle to perform cross-frame analysis, causing many models to disregard the temporal patterns of individual signs. CorrNet mitigates this issue by using specific correlation and identification modules. Essentially, correlations are assigned and charted between frames according to similarity to adjacent frames; this provides contextual temporal information that allows accurate predictions of specific spatial and bodily movements within individual signs [11].

2.2 Facial Occlusion

Sign languages notably incorporate both hand movements and facial expressions, therefore requiring any automated translation software to take both features into account simultaneously. Within the context of Arabic-based sign languages, an obstacle arises due to the prevalence of facially occluding headwear, such as niqabs. Niqabs and similar clothing are commonly worn within Arabic culture and tradition, especially among Muslims, and often cover the whole face, except for the eyes. A weakness of many existing CSLR models is their inability to accurately recognize signs when the facial features of the signer are partially or entirely obscured [10].

2.3 Word Error Rate

Word error rate (WER) is a metric used to measure a model's effectiveness at recognizing and interpreting sign language.

> It measures the difference between the predicted sequence and the ground truth by calculating the minimum number of edit operations (insertions, deletions, and substitutions) required to transform one sequence into the other; hence, lower WER indicates better performance [9].

To monitor the model's progression throughout training, we utilized a development word error rate (dev WER). The dev WER evaluates a model against an example development set at the end of each training epoch; because the actual model never trains on this dataset itself, the dev WER produces an observable and easily comparable figure at each step of training [12]. This prevents the model from parroting gloss translations directly from trained data, requiring it to engage with phrases that it is not already trained on. Essentially, higher dev WER indicates poorer performance.

2.4 Connectionist Temporal Classification (CTC) Decoder

A CTC algorithm temporally connects the input and output of a model when the two timings and alignments are unknown or differ. A sign language translation-to-text system runs the risk of temporal misalignment, meaning the video frame gloss sequences do not match, often because the number of input frames exceeds the number of output words. Since there are more inputs than outputs, the model may fail to label extra frames. The CTC algorithm allows extra frames to be absorbed, using blanks to separate consecutive appearances of the same gloss so that they are not mistakenly combined into a single translation [13].

Occasionally, the CTC may under-train the feature extractor. This occurs when the alignment module finishes learning before the feature extractor, leading to insufficient information relay. The Visual Alignment Constraint (VAC) solves this issue by providing two additional losses, ConvCTC and SeqKD, supervising the feature extractor further [14].

2.5 Loss

The training loss for our model combines three terms that follow CorrNet configurations according to the VAC framework [11, 14]. These terms are SeqCTC, ConvCTC, and Sequence Knowledge Distillation (SeqKD). SeqCTC is a standard CTC loss applied to the model's final output, training the model to produce proper gloss sequences. ConvCTC applies a similar CTC loss earlier in the training process, applied directly on the convolutional branch's predictions before sequence modeling. Min et al. introduced this method because visual feature extractors often under-train when the final output is considered solely with the SeqCTC loss [14]. Finally, SeqKD connects the convolutional and sequence branches together. The distillation is one-directional; the reference is detached such that the gradient only flows through the convolutional branch, effectively aligning the frame distribution of the convolutional branch to the sequence branch [15].

For our adapted model configuration, SeqCTC and ConvCTC are each weighted at 1.0. The SeqKD term utilizes a base weight of 25.0, but is scaled by a warm-up factor over the first three epochs. This results in an effective, steady weight of 7.5 for the signer-independent (SI) configuration and 7.5 for the unseen-sentence (US) configuration (SI and US configurations are detailed in section 3).

3 Data Collection

Due to the nature of sign language, much of our data collection was focused on locating suitable video-extracted images of signers. Cultural and religious norms set Saudi Sign Language (SSL) apart from other sign languages because of its

comparatively decreased emphasis on the face. The Isharah dataset was an ideal choice because it includes video clips from various signers, many wearing facially occluding headwear, which provided differentiated training and observation of model accuracy during development.

Isharah is a large-scale dataset aimed towards assisting in the development of SSL recognition and translation training, built upon tens of thousands of SSL recordings and including a diverse collection of 2000 of the most commonly used sentences among deaf signers [9]. 18 different signers were recruited to carefully record the entirety of the Isharah dataset, promoting the generalizability of the model, and several signers were wearing masks or niqabs, ensuring the presence of facial occlusion as a variable in the model's efficacy. Isharah is segmented into distinct 500, 1000, and 2000-sentence cohorts, providing smaller-scale sets for use [16].

We opted for the Isharah-1000 dataset for several reasons, including its expansive, gloss-based catalog of sentences. However, due to unforeseen errors, a small number of clips were excluded due to missing frame directories. For US configurations, 203 out of 13,500 trained, 14 out of 750 dev, and 10 out of 750 test clips were excluded. For SI configurations, 227 of 10,000 trained were excluded, while the other categories ran smoothly. We did not use the Isharah-2000 or 500 datasets because Isharah-2000 was not available as open source, and Isharah-500 included the same words and phrases as Isharah-1000 (the larger dataset was preferred).

There are two primary pathways within sign language translation systems: unseen-sentence and signer-independent. Unseen-sentence evaluations assess a model's capability to understand and translate novel linguistic combinations of words and phrases by analyzing the movements and signs of known signers. Alternatively, signer-independent systems emphasize accurately translating signs from unseen signers. Although unseen-sentence evaluations have positive benefits, signer-independent training was preferred for this project because it more closely addresses the primary concern of recognizing and differentiating signs from occluded signers.

The videos were formatted as pre-extracted 256x256 JPEGS; the flexibility of this format allows multilateral use, as well as compatibility with CorrNet. Many clips were recorded on smartphones and within regular living conditions, causing variations in quality and other characteristics, although all were recorded at 30 frames per second. Of the 18 signers, 5 wore some form of occluding headwear. Another important detail to note is that the dev WER was tested against signer 02, who is not an occluded signer [9].

4 Methods

Two variants of the CorrNet model were trained against the same Isharah setup in order to ground comparisons between the published CorrNet and our modifications. The stock CorrNet was reproduced and tuned during the training process until it reached a stable working baseline, deemed successful only once results were within one point of the word error rate (WER) (see section 2.3) initially determined by

Alyami et al. Additionally, we addressed two bugs in the stock CorrNet, reducing the WER further. This modified CorrNet was used as our baseline reference point.

The second model, dubbed the 'FaceBranch,' was also grounded in the foundational CorrNet model, but augmented with a parallel stream alongside the original, in which a pose-derived eye region crop (see section 4.5) passed through a small CNN and was fused into the recognizer prior to the final classifier. The design of the second model served to emphasize the eye region of occluded signers, as they become increasingly crucial with the presence of facial coverings over the mouth and other anterior facial elements. This model was deemed successful when the dev WER (see section 2.3) fell below what was observed for the baseline reference (roughly 2 percentage points).

4.1 PyTorch

The primary deep learning framework used in this project was PyTorch, written in Python. PyTorch was used to reproduce the baseline CorrNet model and implement the model modified with a late fusion FaceBranch.

4.2 ORCA

Computational resources used in this work were supported by the Oregon Regional Computing Accelerator (ORCA), funded by NSF CC*-2346732. This cluster was integral in the collection of data and development of our model.

4.3 CTCDecode

During evaluation, gloss probabilities produced by the model were converted into gloss sequences using connectionist temporal classification (CTC) beam search decoding. An open-source CTC beam search decoder was implemented in C++; no prebuilt package of the library existed for our software environment, so it was instead compiled on the cluster. Decoding applied a beam width of 10 with the standard CTC blank symbol; for each video, the highest-scoring beam was retained, and the resulting indices were mapped to gloss labels through the dataset's gloss dictionary. The same decoding procedure was applied to the development evaluations during training and the test evaluation [17].

4.4 Reproducing CorrNet

CorrNet was our baseline CSLR foundation and was trained on the Isharah-1000 benchmark under both the unseen-sentences and signer-independent splits [9, 11]. Training was performed on a single NVIDIA L40S GPU. The model was optimized with Adaptive Moment Estimation (ADAM) with a base learning rate of 0.0001 for 40 epochs.

However, the published training configuration (FP16 mixed precision with dynamic loss scaling, no gradient clipping, and the distillation term active at full weight from the first step) failed to train on the Isharah dataset. More specifically, the runs overflowed in FP16 or collapsed onto a small set of frequent glosses. Further investigation showed that with stock configurations, CorrNet could not memorize 100 clips, which proved that the failure to train stems from the configuration and not the dataset. Therefore, three changes were made to reproduce the published WER of CorrNet:

1. The training was run in full FP32 without automatic precision and with gradient-norm clipping at 5.0.
2. The distillation weight is linearly warmed up from zero over the first 3 epochs, with a distillation cap for both SI and US at 0.30.
3. The memory constraints in FP32 limit the batch size to 2, and for the SI runs, an 8-epoch test revealed that taking an optimizer step every batch outperformed gradient accumulation (∼5 points lower dev WER at similar epochs); therefore, no accumulation is used. The US runs were kept the same because they ran better than the accumulation-free run during our testing.

Certain configurations within the original CorrNet model disabled correlation branches or corrupted gradients that trained on them, resulting in inconsistencies for our FaceBranch. Two main debugs to the CorrNet code allowed us to train on a more consistent model:

1. PyTorch dispatches the CTC loss to either a native kernel or NVIDIA's cuDNN kernel, depending implicitly on the types and devices of its arguments, and the released CorrNet trains through the cuDNN's kernel by default [18]. The unreliability of the default kernel's backward pass under `zero_infinity=True` has been documented in PyTorch's issue tracker, where disabling the cuDNN backend was identified as a solution [19]. Therefore, by forcing the native kernel, the steady-state gradient norm is reduced from a median of 4.9×10^{3} to a standard range between 24 and 60.
2. CorrNet's correlation modules are attached through a learnable gate initialized to zero, and the released configuration specifies `ADAM` with `weight_decay` = 10^{-4}. However, these two combinations appeared destructive due to `ADAM` rescaling each update by the typical gradient and the `weight_decay` parameter approaching zero. The test runs revealed that if the module survives by chance on epoch 0, the correlation branches stay alive. With the correlation branches dead, CorrNet

would lose the feature that distinguishes it from a generic ResNet-BiLSTM recognizer [11] (CorrNet's foundation), and relying on chance is inherently unreliable; therefore, it was decided to place the gate parameters and the correlation weights in separate parameter groups, with the weight decay disabled.

Results from these modifications are outlined in the table below.

Table 1 A table of comparison for code implementations we made to stock CorrNet, and a comparison to the published WERs in the Isharah paper [9]. All test runs use the SI split, dist. cap 0.30, and seed 0. We count the correlation branch to be alive, under the label "Corr. alive", if $|\alpha_i| > 10^{-20}$. The table below shows that the two fixes together lower the Dev-WER and test WER by a significant amount.

Run	**Condition**	**CTC fix**	**Corr. fix**	**Corr. alive**	**Dev WER (%)**	**Test WER (%)**
run12	neither	—	—	3/3*	28.55	40.26
run41	corr. gate only	—	yes	3/3	22.79	40.05
run39	CTC only	yes	—	0/3	23.04	39.95
run26	**both**	yes	yes	3/3	**18.23**	**24.63**
Alyami et al.	published WERs	—	—	—	18.80	31.90

*run12 had all three of the correlation branches alive at the end of the training. As mentioned earlier, both run12 and run41 survived epoch 0 by chance.

There were no other changes or additions made; configurations for run26 (see table 1) will be used for the rest of the paper as a control comparison for the FaceBranch.

4.5 MediaPipe Detection

MediaPipe is an open-source framework from Google that provides real-time perception pipelines, primarily aiding in facial and postural detection and recognition [20]. MediaPipe was applied in the front end of our model to automatically extract cropped images of faces within our training data; conventional facial recognition models within MediaPipe, such as BlazeFace, are limited by their reliance on facial landmarks (e.g., eyes, nose, ears, and lips) [21]. Therefore, we adopted BlazePose, which uses the entire body and head geometry as a reference to constrain the eye positions, tracking and following details across frames through sequential processing. [22]

Since occluded signers still present facial features through the eyes and the upper face region, we hypothesized that extracting and using the localized eye region would be beneficial in improving the WER of CorrNet; using BlazePose, the eye region detection rate for occluded signers was successfully determined to be roughly 88.4% (see figure 1).

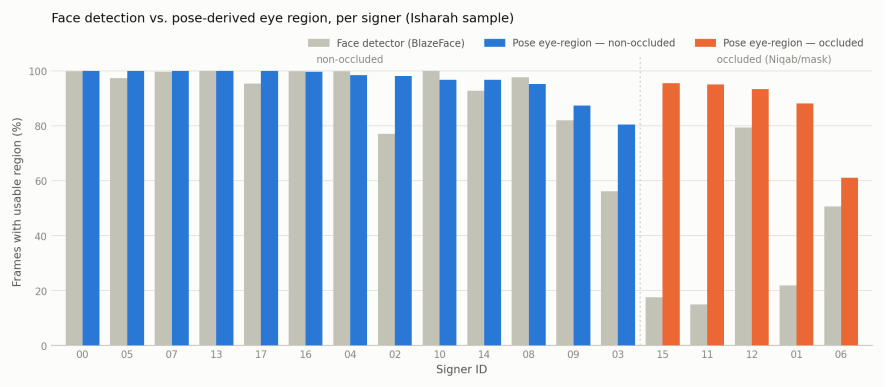


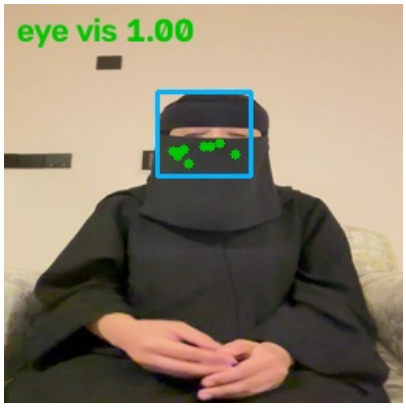
Fig. 1 Compares the detection rate between BlazeFace and BlazePose on occluded and nonoccluded signers separately.

While testing BlazePose, we noticed that eye landmarks frequently drift to the fabrics of niqabs (see image 2a); the parameters in table 2 account for this downward drift. The entire face, rather than solely the eye region, was cropped deliberately, as we hypothesized it would be beneficial in improving the translation WER of non-occluded signers. Additionally, pose-guided cropping of face and hand regions has been used in multi-cue CSLR architectures [23]. All empirical parameters utilized for this spatial and temporal tracking are detailed in table 2.

Table 2 Crop geometry and temporal tracking parameters.

Symbol	Meaning	Value
m	horizontal margin	0.25
γ	height/width aspect	0.9
β	fraction of height above centroid	0.70
$v_{\min}$	eye-visibility acceptance threshold	0.5
ρ	EMA smoothing weight	0.3
K	max temporal hold (frames)	15
$s_{\min}$	min landmark span (px)	4

The resulting crop box is shown in Figure 2, where the eye landmarks are lower than where the eyes are actually located.



(a) MediaPipe detecting a face with its confidence.



(b) The cropped image that will be input to the FaceBranch (see section 4.6.1)

Fig. 2 Represents a visual of what our parameters do and the resulting cropped image. Figure 2a is an image from the Isharah-1000 dataset [9].

The extraction is run once offline and cached as a compressed `.npz` file for every clip, which can be accessed through GitHub [1]. Each file contains the crop corners (x_0, y_0, x_1, y_1), the confidence of facial detection, and the source of the frame (fresh detection, temporal hold, or static fallback). The successful localization of the eye region for occluded signers is then used to improve WER.

4.6 FaceBranch Architecture

This section describes the additions made onto stock CorrNet; as previously mentioned, our FaceBranch crops the faces of signers and runs parallel to stock CorrNet, applying a late fusion architecture to the existing CorrNet model to supplement processing through an eye-crop-focused lens. Figure 3 shows a visual representation of the FaceBranch architecture.

[1] GitHub can be accessed at `https://github.com/aa1152/ALTREU`

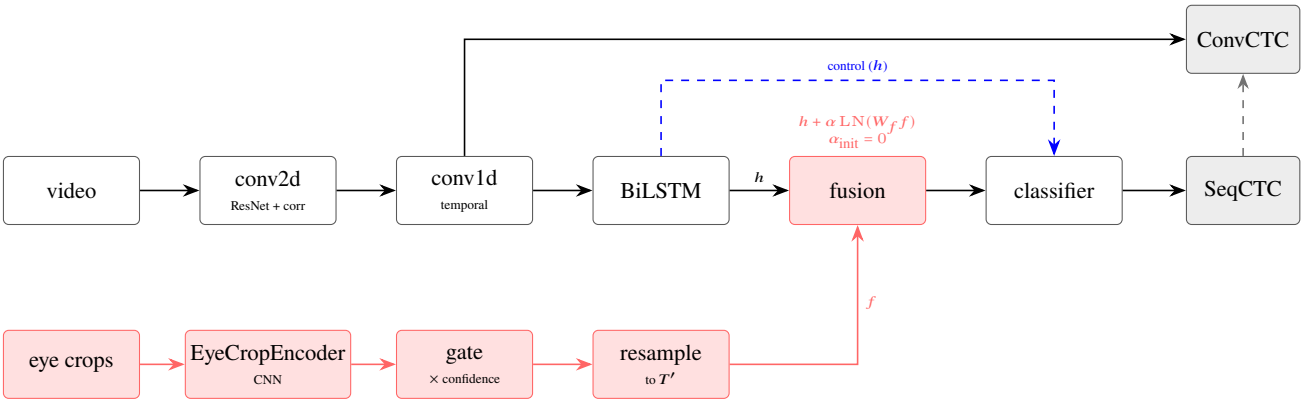


Fig. 3 Highlighted in red are the additions made to the stock CorrNet, running the FaceBranch in parallel with stock CorrNet. Both arms are identical at initialization, and the classifier is shared between both sequence and convolutional heads; self-distillation aligns the two together, so gradient flows only into the convolutional head. Each model was trained at five seeds {0,...,4}. The dashed blue arrow shows the path for the control train, which was also trained on five seeds {0,...,4}. Reference figure 3 for the entirety of section 4.6.

4.6.1 Eye Crops

Frames are processed in parallel through both the base CorrNet model and our FaceBranch, but specific inputs and outputs differ between the two. Specifically, the FaceBranch accepts coordinates of eyecrops within each 256×256 frame as input; these cropped images are subsequently resized to 112×112. For each cropped frame t, there is numerical confidence for successful eye-region detection $c_t \in [0, 1]$ (recall section 4.5); the image from figure 2b is an example of what the input frame looks like for FaceBranch before being resized.

4.6.2 EyeCropEncoder

Cropped frames are subsequently channeled into a small Convolutional Neural Network (CNN), where the EyeCropEncoder has channel widths of 32, 64, 128, and 256. Following standard practices, each block applies a 3×3 convolution of stride 2, followed by batch normalization and a rectified linear activation. Global average pooling followed by a linear projection returned $e_t \in \mathbb{R}^{128}$.

4.6.3 Confidence Gate

After the frames are simplified, each encoded feature is multiplied by its detection confidence.

$$\tilde{f}_t = c_t \, e_t, \tag{1}$$

Because temporal hold and static fallback frames inherently carry reduced confidence levels, we chose to reduce the influence of unreliable frames.

4.6.4 Resampling

The EyeCropEncoder and confidence gate process operate at the original recorded frame rate of 30 fps. However, an issue arises when the base CorrNet passes through `conv1d` block, which contains pooling layers and therefore shrink time factors. If the variables T and T' represent sequence lengths before and after CorrNet's downsampling, the FaceBranch features are linearly interpolated temporally to the length T', denoted with the output f (see figure 3). This matches the output of the sequence head h to temporally align parallel outputs at fusion. More specifically, $f \in \mathbb{R}^{T' \times 128}$.

4.6.5 Late Fusion

The late fusion process is required to align and merge both into a single output [24]. Following the FaceBranch and CorrNet time step, τ is aligned through resampling so that τ represents the same sequence step in both f and h.

Writing $h_\tau \in \mathbb{R}^H$, $H = 1024$, for CorrNet's sequence feature at step τ,

$$h'_\tau = h_\tau + \alpha \cdot \mathrm{LN}(W_\mathrm{f} f_\tau + b_\mathrm{f}), \qquad W_\mathrm{f} \in \mathbb{R}^{H \times d},\ b_\mathrm{f} \in \mathbb{R}^H,\ \alpha \in \mathbb{R}, \tag{2}$$

where LN is the normalization of the layer over the H characteristic dimensions [25]. Then, the gate is initialized to zero,

$$\alpha^{(0)} = 0, \tag{3}$$

so that $h'_\tau = h_\tau$ exactly at initialization and the augmented model is identical to the stock CorrNet before any training step, so any change in WER can be attributed to the FaceBranch. No other changes or additions have been made, and it follows the original CorrNet architecture after fusion.

4.6.6 Control Run

We ran a control run that consists of the debugged stock CorrNet to see how much improvement the FaceBranch makes. The control run follows the original CorrNet architecture and is highlighted by the blue dashed arrow in Figure 3. The resulting comparison can be accessed in section 4.7.

4.7 Results

This section outlines WER results across multiple training trials, where each trial contains an evaluation of the full model alongside a control. Results are displayed for the 4 signers in the evaluation phase, where only signers 11 and 12 wear some form of occluding headwear (specified in the table 3 caption). Overall, a meaningful improvement was observed in WER when comparing our model to the control. δ denotes face - control.

Table 3 All individual signer testing for WER (%) on the Isharah-1000 SI framework. This table includes the FaceBranch arm and the control arm (without FaceBranch) over five seeds. Both arms utilize our debugged CorrNet (section 4.4) with a distillation cap of 0.30. Signer 11 wears a niqab; signer 12 wears a face mask.

Signer	**Arm**	**Seed**					**Mean ± sd**	**Δ**
		0	**1**	**2**	**3**	**4**		
11 *(occluded)*	Face	43.57	44.72	43.86	45.45	44.36	44.39 ± 0.74	−2.65
	Control	43.22	45.07	46.78	50.92	49.23	47.04 ± 3.10	
12 *(part. occl.)*	Face	22.73	23.15	25.78	22.50	22.57	23.35 ± 1.38	−3.63
	Control	26.01	24.78	25.07	31.76	27.28	26.98 ± 2.84	
Clear (08,10)	Face	15.40	14.54	18.20	14.18	15.30	15.52	−2.69
	Control	14.66	23.93	15.78	19.28	17.43	18.22	
Occluded (11,12)	Face	33.15	33.93	34.81	33.97	33.46	33.86	−3.15
	Control	34.61	34.92	35.92	41.34	38.25	37.01	
Overall	Face	24.27	24.22	26.50	24.07	24.37	24.69 ± 1.02	−2.91
	Control	24.63	29.42	25.84	30.30	27.83	27.60 ± 2.38	

Although FaceBranch did not outperform the baseline model in every tested seed, there was a mean average improvement for both occluded and nonoccluded signers, and lower standard deviations as well. However, it is important to note that this result depends on distillation capped at 0.30 for signer-independent sets; no concrete conclusions can be drawn for US or different configurations. We were not able to test for significance due to the time constraints that we had on running more test runs.

4.8 Baseline Comparisons

In order to accurately quantify the strength and efficacy of this model, it must first be compared with contemporary models to determine viability.

This could have been performed by running the Isharah-1000 dataset through other models, or running the datasets of other models through FaceBranch. Due to the importance of the presence of facial occlusions within the dataset, we opted for the former.

With both models applying the same Isharah-1000 dataset, we can accurately compare each model's ability to translate signs from occluded signers. Although general translation capabilities for nonoccluded signers cannot be directly compared, this serves as a solid proof-of-concept for the efficacy of the FaceBranch and eyecrop-focused approach.

Table 4 WER (%) on Isharah-1000 for the signer-independent (SI) and unseen-sentences (US) protocols. All methods use RGB input; the best value per column is in **bold**. *Reported in the Isharah benchmark [9].

Method	Signer-Independent		Unseen-Sentences	
	Dev	**Test**	**Dev**	**Test**
VAC*	18.9	31.9	57.0	49.6
SMKD*	18.5	35.1	56.6	48.0
TLP*	19.0	32.0	70.8	63.3
SEN*	19.1	36.4	66.2	57.3
CorrNet*	18.8	31.9	63.7	55.0
Swin-MSTP*	**17.9**	26.6	73.5	66.1
SlowFastSign*	19.0	32.1	65.5	56.2
This work				
CorrNet (control)	20.02 ± 2.14	27.60 ± 2.38	54.47	**46.58**
CorrNet + FaceBranch	18.48 ± 0.99	$\mathbf{24.69 \pm 1.02}$	**54.33**	**46.58**

For SI, CorrNet + FaceBranch performed the best in the WER test among the models listed in the table. Swin-MSTP outperforms for SI dev WER, but because the dev WER is tested on signer 02 (see section 3), it does not necessarily show that Swin-MSTP outperforms FaceBranch on improving WER for occluded signers. For US, it also produced the lowest WER, although this result isn't decisive because FaceBranch and control incidentally returned the same WERs.

5 Conclusion

Our paper built upon the existing CorrNet model, supplementing it with our FaceBranch through a late-fusion architecture to provide accurate glossed translation for occluded signers. When comparing our model with the baseline CorrNet using the Isharah-1000 dataset, FaceBranch successfully improved the recognition and translation of occluded signers.

5.1 Costs

In attempting to improve the efficacy of existing models, as this model tries to do for CorrNet, it is necessary to consider additional costs and burdens imposed by any additional changes.

Table 5 Model-side cost of the FaceBranch, measured on an NVIDIA L40S in FP32. Forward figures are per clip at batch size 1 on the dataset-mean clip length of 233 frames. The training peak is at batch size 2, which is the actual training configuration.

	Control	**Face**	**Δ**
Trainable parameters	32,749,602	33,305,539	+555,937 (+1.70%)
GFLOPs / clip	1036.3	1057.8	+21.5 (+2.1%)
MFLOPs / frame	4447.8	4540.1	+92.3 (+2.1%)
Forward ms / clip	75.4	76.5	+1.1 (+1.4%)
Peak training memory	18.9 GB	19.5 GB	+646 MB

The table shows that FaceBranch does not add much cost compared to the control. Additionally, in terms of training costs, GPU-h varied at an average of less than 0.20 for 40-epoch runs between the face and control arms, negligible compared to 0.79 GPU-h per single epoch. The FaceBranch was consistently longer in all cases, but only by fewer than 15 minutes over 20 to 40 hour runs.

The primary difference is in the creation and alignment of eye crops within the FaceBranch. Across 8 shards, the MediaPipe eyebox summed to 35.6 CPU-hours across 14,773 clips and 3.44 million frames. This amounted to 61 MB of box metadata overall. However, this processing was performed early in the training of our model, and optimizations could significantly boost the speed of these calculations. Although model training and processing all require marginally longer times and greater energy consumption for our FaceBranch, we determined that the bulk of our model's cost is in developing and creating the eye crops.

5.2 Applications

The successful implementation and integration of facial features in enhancing sign language translation provides several immediately relevant applications.

This analysis serves as a foundational proof-of-concept to be expanded upon, both in complexity and breadth of language databases. Although our initial goal was to improve the translation availability for Arabic Sign Language signers, the ability to draw contextual information from general facial features such as eyes or eyebrows can be applied to nonoccluded signers and improve general sign language translation in all capacities.

5.2.1 Other Forms of Occlusions

Even in non-SSL signing countries or countries with smaller populations wearing niqabs and hijabs, our model's flexible approach could just as easily be implemented to improve translation detection. Furthermore, this model can aid in the process of sign-language recognition and interpretation from those wearing masks, scarves, large coats, and any other kind of occluding facial or outerwear. This proves crucial in places and times where these facial occlusions, such as masks, may be required, including but not limited to medical and controlled research settings.

5.2.2 Knowledge Distillation

Using speech recognition as a point of comparison, knowledge distillation (KD) has been used in the past to improve model efficiency and recognition accuracy [26]. The same principle can be applied to AI-powered smart glasses for interpreting sign language. Furthermore, KD also serves to reduce overall processing intensity and maintain performance within edge devices and other external hardware [27]. Since CorrNet itself is cost heavy, KD would make it easier to integrate the model into AI smart glasses. Future refinements in training the model may involve applying knowledge distillation to streamline its integration with AI glasses for smooth, consistent translation performance in real-world applications.

5.3 Shortcomings and Next Steps

During training and model creation, several shortcomings constrained the development and output of our model.

Verifying eyecrop positioning was done automatically, but occasionally required manual adjustments due to fabric-focused drifting. Future models could work towards phasing out this manual verification to improve training efficiency. Furthermore, this

shortcoming may cause landmark-anchored crops to inherit the same drift throughout training.

Additionally, person detection tended to degrade in accuracy when the signer's entire body was hidden and the head was occluded; although this was not immediately relevant for the Isharah dataset, this may become an obstacle in real-world applications with AI-powered glasses.

Acquiring data solely from a single dataset such as Isharah provides benefits in video quality and accuracy, but is bound to suffer from small trends, biases, and areas of concern. Despite the dataset's variance, there are likely regional differences or underrepresented countries because of the wide-ranging influence of SSL. Future model building and training could be improved upon by incorporating more varied datasets and sources.

Furthermore, although we compared results with state-of-the-art RGB models, this was not done for POSE-based models because our model was based in RGB. This ultimately lowers the scope of our comparisons and results in uncertainty regarding whether or not our model can surpass POSE-based models. In addition, lack of time and resources limited comparisons against Phoenix-14 and Phoenix-14T, two of the most widely used German-based sign language datasets within other models [28]. Lastly, FaceBranch was built upon an older version of CorrNet; therefore, further improvements using CorrNet+ have yet to be explored [29].

Acknowledgements The authors declare that generative AI was used for assistance in code development, LaTeX formatting of tables from author produced results, and copy editing for grammar and clarity. All results, analysis, and conclusions were made and verified by the authors.

References

[1] "Deafness and hearing loss." [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] J. O. Wobbrock, "Situationally-Induced Impairments and Disabilities," in *Web Accessibility: A Foundation for Research*, Y. Yesilada and S. Harper, Eds. London: Springer, 2019, pp. 59–92. [Online]. Available: https://doi.org/10.1007/978-1-4471-7440-0_5

[3] O. Crasborn, "Nonmanual Structures in Sign Language," *Encyclopedia of Language and Linguistics*, vol. 8, Dec. 2006.

[4] H. Luqman, "SignVLM: a pre-trained large video model for sign language recognition," *PeerJ Computer Science*, vol. 11, p. e3112, Sep. 2025. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12453763/

[5] A. Yin, T. Zhong, L. Tang, W. Jin, T. Jin, and Z. Zhao, "Gloss attention for gloss-free sign language translation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 2551–2562. [Online]. Available: http://openaccess.thecvf.com/content/CVPR2023/html/

Yin_Gloss_Attention_for_Gloss-Free_Sign_Language_Translation_CVPR_2023_paper.html

[6] T. Tao, Y. Zhao, T. Liu, and J. Zhu, "Sign Language Recognition: A Comprehensive Review of Traditional and Deep Learning Approaches, Datasets, and Challenges," *IEEE Access*, vol. 12, pp. 75 034–75 060, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10526274

[7] X. Geng, Y. Li, Z. Ma, Z. Lu, and Q. Miao, "No blind alignment but generation: A different view of continuous sign language recognition based on diffusion," *Pattern Recognition*, vol. 169, p. 111960, Jan. 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S003132032500620X

[8] N. Algethami, R. Farhud, M. Alghamdi, H. Almutairi, M. Sorani, and N. Aleisa, "Continuous Arabic Sign Language Recognition Models," *Sensors*, vol. 25, no. 9, p. 2916, May 2025. [Online]. Available: https://www.mdpi.com/1424-8220/25/9/2916

[9] S. Alyami, H. Luqman, S. Al-Azani, M. Alowaifeer, Y. Alharbi, and Y. Alonaizan, "Isharah: A Large-Scale Multi-Scene Dataset for Continuous Sign Language Recognition," *IEEE Transactions on Multimedia*, vol. 28, pp. 5050–5058, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11397217

[10] S. Elhassen, L. Khuzayem, A. Alhothali, O. Alzamzami, and N. Alowaidi, *Continuous Saudi Sign Language Recognition: A Vision Transformer Approach*, Sep. 2025.

[11] L. Hu, L. Gao, Z. Liu, and W. Feng, "Continuous Sign Language Recognition with Correlation Network," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Vancouver, BC, Canada: IEEE, Jun. 2023, pp. 2529–2539. [Online]. Available: https://ieeexplore.ieee.org/document/10205442/

[12] A. Y. Ng, "Preventing "Overfitting" of Cross-Validation Data," in *Proceedings of the Fourteenth International Conference on Machine Learning*, ser. ICML '97. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Jul. 1997, pp. 245–253.

[13] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proceedings of the 23rd international conference on Machine learning*, ser. ICML '06. New York, NY, USA: Association for Computing Machinery, Jun. 2006, pp. 369–376. [Online]. Available: https://dl.acm.org/doi/10.1145/1143844.1143891

[14] Y. Min, A. Hao, X. Chai, and X. Chen, "Visual alignment constraint for continuous sign language recognition," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 11 542–11 551.

[15] A. Hao, Y. Min, and X. Chen, "Self-Mutual Distillation Learning for Continuous Sign Language Recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 11 303–11 312. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2021/html/

Hao_Self-Mutual_Distillation_Learning_for_Continuous_Sign_Language_Recognition_ICCV_2021_paper.html

[16] F. M. E. Elden, "CSLRConformer: A Data-Centric Conformer Approach for Continuous Arabic Sign Language Recognition on the Isharah Datase," Aug. 2025, arXiv:2508.01791 [cs.CV]. [Online]. Available: http://arxiv.org/abs/2508.01791

[17] "parlance/ctcdecode," Jul. 2026, original-date: 2017-06-14T20:17:41Z. [Online]. Available: https://github.com/parlance/ctcdecode

[18] "CTCLoss — PyTorch 2.13 documentation." [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.CTCLoss.html

[19] "Backward pass fails due to CTCLoss in case zero_infinity=True and torch.backends.cudnn.enabled=True · Issue #49046 · pytorch/pytorch." [Online]. Available: https://github.com/pytorch/pytorch/issues/49046

[20] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A Framework for Building Perception Pipelines," Jun. 2019, arXiv:1906.08172 [cs.DC]. [Online]. Available: http://arxiv.org/abs/1906.08172

[21] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," Jul. 2019, arXiv:1907.05047 [cs.CV]. [Online]. Available: http://arxiv.org/abs/1907.05047

[22] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device Real-time Body Pose tracking," Jun. 2020, arXiv:2006.10204 [cs.CV]. [Online]. Available: http://arxiv.org/abs/2006.10204

[23] H. Zhou, W. Zhou, Y. Zhou, and H. Li, "Spatial-Temporal Multi-Cue Network for Continuous Sign Language Recognition," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, pp. 13 009–13 016, Apr. 2020. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/7001

[24] K. Gadzicki, R. Khamsehashari, and C. Zetzsche, "Early vs Late Fusion in Multimodal Convolutional Neural Networks," in *2020 IEEE 23rd International Conference on Information Fusion (FUSION)*, Jul. 2020, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9190246

[25] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," Jul. 2016, arXiv:1607.06450 [stat.ML]. [Online]. Available: http://arxiv.org/abs/1607.06450

[26] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge Distillation: A Survey," *International Journal of Computer Vision*, vol. 129, no. 6, pp. 1789–1819, Jun. 2021, arXiv:2006.05525 [cs.LG]. [Online]. Available: http://arxiv.org/abs/2006.05525

[27] A. Alkhulaifi, F. Alsahli, and I. Ahmad, "Knowledge distillation in deep learning and its applications," *PeerJ Computer Science*, vol. 7, p. e474, Apr. 2021. [Online]. Available: https://peerj.com/articles/cs-474

[28] J. Forster, C. Schmidt, T. Hoyoux, O. Koller, U. Zelle, J. H. Piater, and H. Ney, "RWTH-PHOENIX-weather: A large vocabulary sign language recognition and translation corpus." in *LREC*, vol. 9, 2012, pp. 3785–3789. [Online]. Available: http://www-i6.informatik.rwth-aachen.de/publications/download/773/forster-lrec-2012.pdf

[29] L. Hu, W. Feng, L. Gao, Z. Liu, and L. Wan, "CorrNet+: Sign Language Recognition and Translation via Spatial-Temporal Correlation," Apr. 2024, arXiv:2404.11111 [cs.CV]. [Online]. Available: http://arxiv.org/abs/2404.11111